

SemaFailBench

Detecting Silent Semantic Degradation in LLM Serving

Semantic Canary-Based Monitoring for Deployment-Time Faults

Rataj Albalawi,
Shaden Alangari
Remas Almarri

Problem/Motivation

An LLM deployment can remain operational while its semantic behavior degrades.

Deployment / Configuration Fault



Service still operational + Infrastructure appears healthy



Semantic behavior degrades



SILENT FAILURE

Research Gap/ If we already have LLM benchmarks and infrastructure monitoring, why do we need SemaFailBench?

LLM Fault / Reliability Studies

Show that quantization, templates, model updates, etc. can change behavior

→ **but usually evaluate the effect offline**

LLM Benchmarks

Measure instruction following, safety, factuality, grounding, etc.

→ **but are designed to evaluate capability, not deployment health**

SRE / Infrastructure Monitoring

Detects crashes, latency, errors, resource problems and operational failures

→ **but does not directly verify semantic correctness**

No established end-to-end methodology for injecting silent LLM serving faults and detecting their capability-specific semantic degradation using lightweight canary prompts while infrastructure remains healthy.

Literature review

The literature review informed four stages of SemaFailBench design.

① Fault Selection

Deployment & reliability literature

Used to identify:

- **Realistic serving-time failure mechanisms**
- **Evidence of semantic degradation**
- **Silent-failure candidates**

② Capability Selection

LLM evaluation literature

Used to identify:

- Semantic behaviors worth monitoring
- Capability boundaries
- Deterministically measurable behaviors

③ Subtype & Canary Design

Behavioral testing + benchmark literature

Used to define:

- Focused behavioral tests
- One behavior per canary
- Objective expected outcomes
- Prompt diversity

④ Evaluation Design

Monitoring & regression-testing literature

Used to define:

- Healthy vs. faulty comparison
- Baseline-based regression detection
- False alarms

Research objective

RQ1 — Degradation

Do selected deployment-time faults cause measurable semantic degradation while the service remains operational?

RQ2 — Impact

Which semantic capabilities and behaviors are affected by each fault?

RQ3 — Detection

Can lightweight semantic canaries reliably detect these regressions when conventional infrastructure monitoring does not?

How accurately, quickly, and cheaply can semantic canaries detect these failures?

Fault Selection Criteria

Fault Selection Criteria

Deployment Relevance

- ✓ Deployment-time or serving-time fault
- ✓ Accidental operational / configuration error
- ✓ Distinct deployment mechanism

Silent-Failure Requirement

- ✓ Service remains operational
- ✓ Conventional infrastructure alerts remain normal
- ✓ Plausible measurable semantic degradation

Experimental Feasibility

- ✓ Independently injectable
- ✓ Reproducible

Selected Faults

F1 — Quantization Regression

Incorrect / degraded quantized serving configuration

F2 — Model / Checkpoint Version Regression

Wrong or unintended model checkpoint served

F3 — Tokenizer–Checkpoint Mismatch

Tokenizer does not match the served checkpoint

F4 — Chat-Template Mismatch

Incorrect conversation template applied at serving time

F5 — Generation Configuration Drift

Unintended change in decoding / generation settings

F6 — Wrong / Stale LoRA Adapter

Incorrect adapter loaded with the correct base model

Not vllm feasible

F7— Stale Retrieval Snapshot / Version Drift

Retrieval remains operational but supplies outdated information.

F8 — Embedding Model ↔ Vector Index Mismatch

Retrieval executes, but incompatible embeddings can degrade document ranking.

Capability

Cap 1 — Instruction-Following Fidelity

Does the model correctly follow explicit user instructions and constraints?

Cap 2 — Structured-Output Validity & Value Accuracy

Does the model produce the required structure with correct values?

Cap 3 — Factual / Knowledge Recall Accuracy

Does the model correctly recall factual knowledge?

Cap 4 — Safety / Alignment Stability

Does the model maintain appropriate and consistent safety behavior?

Cap 5 — Retrieval-Grounded Response Fidelity

Does the model correctly use and remain faithful to retrieved evidence?

Capabilities were selected to represent distinct, measurable semantic behaviors potentially affected by the identified deployment faults

Fault × Capability

Fault	Cap 1 Instruction Following	Cap 2 Structured Output	Cap 3 Factual Recall	Cap 4 Safety / Alignment	Cap 5 Retrieval Grounding
F1 Quantization Regression	✓	✓	✓	✓	—
F2 Model / Checkpoint Regression	✓	✓	✓	✓	—
F3 Tokenizer–Checkpoint Mismatch	✓	✓	✓	✓	—
F4 Chat-Template Mismatch	✓	✓	—	✓	—
F5 Generation Configuration Drift	✓	✓	—	✓	—
F8 Wrong / Stale LoRA Adapter	✓	✓	✓	✓	—
F6 Stale Retrieval Snapshot	—	—	✓	—	✓
F7 Embedding ↔ Index Mismatch	—	—	—	—	✓

Subtype Derivation Methodology

How did we define the subtypes?

Benchmark Literature

Identify candidate semantic behaviors



Merge Overlap

Combine behaviors measuring the same thing



Remove

Difficulty levels • domain labels • duplicates



Keep only subtypes that are:

- **Distinct** semantic behaviors
- **Supported** by literature
- **Deterministically measurable**
- **Potentially sensitive** to deployment faults
- **Minimally overlapping**

Final Subtypes

Instruction Following

6 subtypes

Structured Output

6 subtypes

Factual Recall

4 subtypes

Safety / Alignment

3 subtypes

Retrieval Grounding

5 subtypes

5 Capabilities → 24 Semantic Subtypes

Canary Design Methodology

1 — Target one behavior

2 — Define expected behavior first

3 — Keep the probe isolated

4 — Make it objectively scoreable

5 — Use realistic wording

6 — Compare Healthy vs. Faulty

150 canaries → 5 capabilities → 24 semantic subtypes

Canary Examples / Dataset

Capability	Subtype	Example of what the canary tests
Instruction Following	Keyword Exclusion	Can the model obey a specific prohibited-word constraint?
Structured Output	Type Strictness	Does a requested field retain the required JSON data type?
Factual Recall	Negative Fact / Misconception	Does the model reject a plausible but false factual premise?

Planned Experiment & Evaluation

① Healthy Baseline

Run 150 canaries repeatedly

Record semantic + infrastructure
baseline

② Inject One Fault

One deployment fault at a time
Keep everything else fixed

③ Measure Semantic Degradation

Compare Healthy vs. Faulty

Overall → **Capability** → **Subtype**

$\Delta = P_{\text{Healthy}} - P_{\text{Faulty}}$

④ Verify Silent Failure

Semantics: × Degraded

Infrastructure: ✓ No conventional alert

⑤ Build & Calibrate Detector

Semantic drift → threshold → **Healthy / Faulty**

Validation → **Freeze threshold** → **Held-out test**

⑥ Evaluate Detection

Accuracy → Recall · FPR · Precision/F1 · AUC

Speed → Queries to detect

Cost → Queries per check

Baseline gap → Semantic vs. infrastructure detection

GPU and Model choice